

Tipos de dados em Python: String

Aprenda neste artigo como usar o objeto String na linguagem Python.

O tipo String é usado principalmente para gravar informações de texto, como o nome de uma pessoa ou uma marca de roupas, por exemplo. Nele temos uma sequência de caracteres que formam uma string. O Python permite formar strings com um par de aspas simples ou duplas. O exemplo do **Código 1 mostra a criação e a exibição de variáveis desse tipo.**

Guia do artigo:

- Índice
- Concatenação de Strings
- Comparação de Strings
- Principais métodos de Strings
- Método find()
- Método replace()
- Método split()
- Método upper()
- Método lower()
- Acentuação no Python

De acordo com a [documentação oficial](#), não existe o tipo `char` no Python, que é caracterizado por apenas um caractere. Caso haja uma variável com apenas um caractere, o tipo dela será `string`.

```
1 nome_1 = "Ricardo Alves"
2 nome_2 = 'Joana Melo'
3
4 print(type(nome_1)) # type 'str'
5 print(type(nome_2)) # type 'str'
```

As linhas 1 e 2 declaram variáveis do tipo `String`. Na primeira linha são usadas aspas duplas, e na segunda, aspas simples. A impressão dos tipos das variáveis feitas nas linhas 4 e 5 vão exibir `str`, que corresponde ao tipo `String`.

Índice

As strings são sequências de caracteres, de forma que podemos acessar um caractere em uma dada posição utilizando um índice. No exemplo a seguir, caso se queira obter o caractere na primeira posição da string `nome`, basta acessar o índice `0` da variável. O acesso e a impressão do valor seriam feitos como no **Código 2**.

```
1 nome = 'Daniel'
2 print(nome[0]) # D
```

Código 2. Acesso e impressão de strings

A variável `nome` com o conteúdo `Daniel` foi acessada no índice `0`, que é por onde começa a cadeia de caracteres. O resultado será o caractere `D`.

Caso tente acessar um índice inexistente, o seguinte erro será exibido: `IndexError :`

`string index out of range`. Isso acontece porque o índice que se tentou acessar está fora do range da cadeia de caracteres da variável. No exemplo dado, a string "Daniel" só nos permitiria acessar até o índice 5. Se tentarmos acessar `nome[6]`, esse erro seria gerado.

Há também a possibilidade de "fatiar" uma variável do tipo String, retornando um "pedaço" dela. No **Código 3** podemos ver essa característica na manipulação de Strings.

```
1 | nome = "Daniel"
2 | print(nome[0:3]) # Dan
```

Código 3. Manipulando String

O resultado do código acima seria a impressão da substring `Dan`. Nela definimos o valor `0` como o início da string que será fatiada até antes da posição que será o limite, que nesse caso é o índice `3`.

Também podemos usar índices negativos para as posições dos caracteres nas strings. Nesse caso, a ordem será inversa, começando do último até o primeiro, como podemos ver no exemplo abaixo:

```
1 | nome = "Daniel"
2 | print(nome[-2]) # e
```

Código 4. Acessando índice de string

O resultado impresso no **Código 4** será o caractere `e`.

Imutabilidade

Uma string no Python é uma sequência de caracteres imutável.

Para ver como isso ocorre na prática, vamos fazer uso da função `id()`, que retorna a identidade de um objeto.

```
1 nome = 'Eduardo'
2 print(id(nome))
3 nome = 'Felipe'
4 print(id(nome))
```

Código 5. Uso da função `id()`

O **Código 5** cria a variável `nome` na linha 1 e depois imprime a identidade dela na linha 2. Já na linha 3 damos um novo valor à variável e novamente imprimimos a sua identidade na linha 4. Os resultados que serão impressos não serão iguais, mostrando que as identidades da variável `nome` são distintas, mostrando um objeto diferente do que foi criado. Então pode-se concluir que a string não foi alterada, mas que foi criada uma nova.

Identidade no Python é o valor de referência do endereço de memória.

Também não é possível alterar o valor de uma determinada posição de uma string. No exemplo abaixo, podemos ver como isso ocorre na prática:

```
1 nome = 'Eduardo'
2 nome[6] = 'a'
```

Código 6. Atribuição de valor na string

No **Código 6** teremos um erro na execução do código, que vai retornar `TypeError: 'str' object does not support item assignment` informando que a string não suporta atribuição de itens, ou seja, não podemos atribuir valor a nenhuma posição de um item na string.

Um recurso importante quando estamos trabalhando com esse tipo é a função `len()`, que retorna o comprimento de uma string. O código abaixo mostra como é o seu uso:

```
1 nome_1 = 'Rodrigo'
2 nome_2 = 'Ana'
3
4 print(len(nome_1)) # 7
5 print(len(nome_2)) # 3
```

Código 7. Obtendo tamanho de string

No **Código 7**, vemos que nas linhas 1 e 2 criamos duas variáveis que recebem textos. Nas linhas 4 e 5 exibimos com a função `len()` o tamanho de cada uma delas. A linha 4 vai retornar o resultado 7, enquanto a linha 5 vai retornar o resultado 3.

Concatenação de Strings

Há casos em que é necessário juntar informações textuais e para esses denominamos concatenação, que é a junção do conteúdo de strings. Vamos fazer um exemplo no qual podemos ver como isso ocorre na prática:

```
1 nome = 'Daniel'
2 sobrenome = 'Silva'
3
4 nome_completo = nome + ' ' + sobrenome
5
6 print(nome_completo) # Daniel Silva
```

Código 8. Junção de strings

Nas linhas 1 e 2 do **Código 8** são criadas as variáveis `nome` e `sobrenome` do tipo String. O processo de concatenar variáveis ocorre na linha 4, na qual a variável `nome_completo` recebe o conteúdo das variáveis declaradas. Para fazer a concatenação entre strings no Python é necessário usar o sinal de adição `+`.

Comparação de Strings

No Python podemos comparar strings de duas formas distintas: com o operador `==` ou `is`.

Com o operador `==` verificamos se o conteúdo de duas strings é igual, como feito no **Código 9**.

```
1 nome_1 = 'Eduardo'
2 nome_2 = 'Eduardo'
3
4 if nome_1 == nome_2:
5     print('iguais')
6 else:
7     print('diferentes')
```

Código 9. Comparação de igualdade

Nas linhas 1 e 2 temos a criação de duas variáveis que têm o mesmo conteúdo, a string `Eduardo`. Na linha 4 temos a comparação entre as strings, que no caso de a condição ser verdadeira, imprimirá a mensagem da linha 5. Caso seja falsa, a mensagem da linha 7 será impressa.

Já com o operador `is`, o que será comparado é a referência do endereço na memória. Vamos usar o mesmo código anterior apenas trocando a condição:

```
1 nome_1 = 'Eduardo'
2 nome_2 = 'Eduardo'
3
4 if nome_1 is nome_2:
5     print('iguais')
6 else:
7     print('diferentes')
```

Código 10. Comparação de identidade

Veja que agora na linha 4 do **Código 10** é usada a condição `nome_1 is nome_2`, o que seria o equivalente a usar `id(nome_1) == id(nome_2)`. A função `id()` retorna a identidade de um objeto na forma de um número inteiro. Trata-se de uma referência a um endereço reservado na memória na criação de um objeto. Quando uma string é criada no Python, é aplicado o conceito de string interning.

Quando criamos uma string no Python, ela é armazenada num lugar da memória não sendo repetida em outro. Todos os objetos que receberem o conteúdo dessa string apontarão para o mesmo endereço de memória. Esse mecanismo diminui o consumo de memória.

Principais métodos de Strings

No Python existem métodos que podem ser usados para fazer operações com strings. Eles podem ser bem úteis em algumas situações. A seguir vemos alguns exemplos.

Os métodos de string retornam novos valores, mas não alteram a string original.

Método find()

Com o método `find()` podemos procurar uma substring dentro de uma string e retornar a posição onde ela foi encontrada, como mostra o **Código 11**.

```
1 | mensagem = 'string no Python'
2 | print(mensagem.find('Python')) # 10
```

Código 11. Método find()

Na linha 2 imprimimos o resultado do método `find()`, que procura pela ocorrência da string `Python` dentro da variável `mensagem`. O resultado será 10.

No caso de a ocorrência não ser encontrada, o resultado será `-1`, como mostra o **Código 12**.

```
1 | mensagem = 'string no Python'
2 | print(mensagem.find('Java')) # -1
```

Código 12. Método find()

Método replace()

O método `replace()` é utilizado para substituir ocorrências de substrings dentro de uma string. O **Código 13** mostra como isso acontece na prática.

```
1 | mensagem = 'Quero aprender Java! Na DevMedia tem salas de Java para apre
2 | nova_mensagem = mensagem.replace('Java', 'Python')
3 | print(nova_mensagem) # Quero aprender Python! Na DevMedia tem salas de P
```

Código 13. Método replace()

Na variável `mensagem`, trocamos todas as ocorrências de `Java` por `Python` utilizando o método `replace()`, e o resultado disso atribuímos a variável `nova_mensagem`, que é impressa na linha 3. O resultado do código retorna: `Quero aprender Python! Na DevMedia tem salas de Python para aprender essa linguagem.`

Método split()

Com o método `split()` desmembramos uma string em múltiplas strings através de um separador passado no parâmetro, retornando todas numa lista.


```
1 mensagem = 'Estou aprendendo Python na DevMedia'
2 nova_mensagem = mensagem.split(' ')
3 print(type(nova_mensagem)) # type 'list'
4 print(nova_mensagem) # ['Estou', 'aprendendo', 'Python', 'na', 'DevMedia']
```

Código 14. Método split()

No **Código 14** criamos a variável `mensagem` na linha 1. Na linha 2 usamos o método `split()` nela e atribuímos isso como valor na variável `nova_mensagem`. O argumento passado como parâmetro no método é um separador, o que fará com que a string seja fragmentada onde ele estiver presente. No exemplo que vimos, o separador usado foi um espaço vazio. Na linha 3 imprimimos o tipo da variável `nova_mensagem` que é uma lista, e na linha 4 imprimimos o valor dela.

É obrigatório o uso de um separador na função `split()` quando usada com aspas, pois caso contrário, um erro será gerado com a mensagem `ValueError: empty separator`. Se as aspas não forem usadas com `split()`, a função considerará o espaço em branco como separador.

Note que a lista retornada pode ser acessada pelos seus índices, como no **Código 15**.

```
1 mensagem = 'Estou aprendendo Python na DevMedia'
2 lista_mensagem = mensagem.split(' ')
3 print(lista_mensagem[1]) # aprendendo
```

Código 15. Método split()

O índice `1` da variável `lista_mensagem` será impresso na linha 3, que exibirá a string `aprendendo`.

No caso de um separador não ser encontrado na string pelo método `split()`, uma lista com apenas um único índice será criada.

Método upper()

Com o método `upper()` retornamos uma cópia da string com todas as letras minúsculas convertidas em maiúsculas. Segue no **Código 16** uma demonstração disso na prática.

```
1 | mensagem = 'eu gosto de Python'
2 | nova_mensagem = mensagem.upper()
3 |
4 | print(nova_mensagem) # EU GOSTO DE PYTHON
```

Código 16. Método upper()

Após criar a variável na linha 1, usamos a variável `nova_mensagem` para receber a variável `mensagem` com todas as letras convertidas em maiúsculo, e na linha 4 a imprimimos.

Método lower()

Com o método `lower()`, retornamos uma cópia da string com todas as letras maiúsculas convertidas em minúsculas. Vejamos o **Código 17**.

```
1 | mensagem = 'eu gosto de Python'
2 | nova_mensagem = mensagem.lower()
3 |
4 | print(nova_mensagem) # eu gosto de python
```

Código 17. Método lower()

Depois de criar a variável na linha 1, usamos a variável `nova_mensagem` para receber a variável `mensagem` com todas as letras convertidas em minúsculo, e na última linha imprimimos o seu valor.

O Python possui diversos outros métodos que podem ser usados dependendo da necessidade exigida num determinado contexto.

Acentuação no Python

Para podermos usar acentuação no Python, devemos definir a codificação utf-8

```
1 | nome = 'João da Silva'
2 | print(nome)
```

Código 18. Erro de acentuação

O código acima irá gerar o seguinte erro: `SyntaxError: Non-ASCII character '\xc3'` que é gerado devido ao caractere acentuado que usamos na variável `nome`.

Para resolver esse problema, podemos usar `#coding: utf-8` no início do código, de acordo com que é instruído na documentação.

```
1 | # coding: utf-8
2 | nome = 'João da Silva'
3 | print(nome)
```

Código 19. Acentuação

Na versão 3 do Python, isso não é necessário.

Nesse artigo vimos como trabalhar com o tipo de dado String. Esse objeto do Python mostra uma gama de opções com o qual podemos trabalhar facilmente com as sequências de caracteres. Com o acesso aos índices e as funções que vimos no decorrer do conteúdo temos recursos que nos ajudam a atender uma grande variedade de situações envolvendo informações textuais.